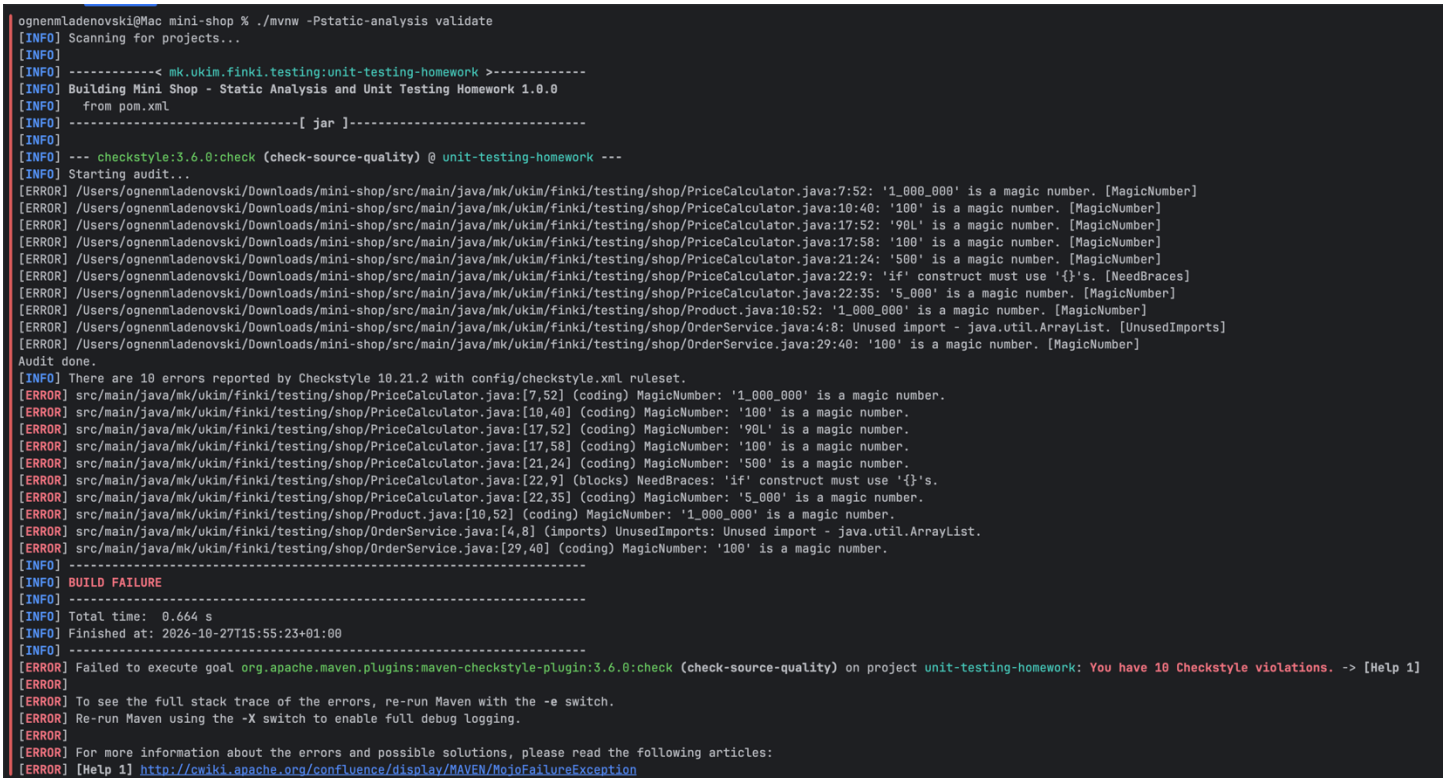
Тестирање на софтвер

Извештај за лабораториска вежба 1

1. Статичка анализа

При извршување на командата **./mvnw -Pstatic-analysis validate**, ми се појавија следните грешки во кодот.



Според ова може да се види дека има: **7 грешки** во класата **PriceCalculator.java**, **1 грешка** во класата **Product.java** и **2 грешки** во класата **OrderService.java.**

1. Грешките во PriceCalculator.java:

6 грешки беа за **MagicNumber**, тоа го поправив со воведување на **public static final** променливи кои што ја содржеа вредноста на променливите кои ја даваа таа грешка:

public static final int ***MAX_UNIT_PRICE*** = 1_000_000;
public static final int ***MAX_QUANTITY*** = 100;
public static final long ***PREMIUM_DISCOUNT*** = 90L;
public static final int ***PERCENTAGE*** = 100;
public static final int ***SHIPPING*** = 500;
public static final int ***FREE_DELIVERY*** = 5_000;

По воведувањето на овие променливи и повторно извршување на командата **./mvnw -Pstatic-analysis validate**, можеме да видиме дека единствената грешка што останува е **NeedBraces**:

```
[INFO] There are 4 errors reported by Checkstyle 10.21.2 with config/checkstyle.xml ruleset.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/PriceCalculator.java:[28,9] (blocks) NeedBraces: 'if' construct must use '{}'s.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/Product.java:[10,52] (coding) MagicNumber: '1_000_000' is a magic number.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[4,8] (imports) UnusedImports: Unused import - java.util.ArrayList.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[29,40] (coding) MagicNumber: '100' is a magic number.
```

Тоа значи дека if-else условот ни фалат загради, и тоа го воведов на следниот начин:

if (discountedSubtotal >= *FREE_DELIVERY*) **{**
  return discountedSubtotal;
}
else {
  return discountedSubtotal + *SHIPPING*;
}

При повторно извршување на тестовите можеме да видиме дека веќе нема грешки во оваа класа

```
[INFO] There are 3 errors reported by Checkstyle 10.21.2 with config/checkstyle.xml ruleset.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/Product.java:[10,52] (coding) MagicNumber: '1_000_000' is a magic number.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[4,8] (imports) UnusedImports: Unused import - java.util.ArrayList.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[29,40] (coding) MagicNumber: '100' is a magic number.
```

Кодот од класата **PriceCalculator.java ПРЕД** промените:

```java
package mk.ukim.finki.testing.shop;

/** Calculates the total for an order containing one product type. All amounts are cents. */
public class PriceCalculator {  2 usages

    public int calculateTotal(int unitPriceCents, int quantity, boolean premiumCustomer) {  1 usage
        if (unitPriceCents < 1 || unitPriceCents > 1_000_000) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (quantity < 1 || quantity > 100) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }

        int subtotal = unitPriceCents * quantity;
        int discountedSubtotal;
        if (premiumCustomer == true) {
            discountedSubtotal = (int) (subtotal * 90L / 100);
        } else {
            discountedSubtotal = subtotal;
        }
        int shipping = 500;
        if (discountedSubtotal >= 5_000)
            shipping = 0;
        return discountedSubtotal + shipping;
    }
}
```

Кодот од класата **PriceCalculator.java ПОСЛЕ** промените:

```java
package mk.ukim.finki.testing.shop;

/** Calculates the total for an order containing one product type. All amounts are cents. */
public class PriceCalculator {  2 usages

    public static final int MAX_UNIT_PRICE =  1_000_000;  1 usage
    public static final int MAX_QUANTITY = 100;  1 usage
    public static final long PREMIUM_DISCOUNT = 90L;  1 usage
    public static final int PERCENTAGE = 100;  1 usage
    public static final int SHIPPING = 500;  1 usage
    public static final int FREE_DELIVERY = 5_000;  1 usage

    public int calculateTotal(int unitPriceCents, int quantity, boolean premiumCustomer) {  1 usage
        if (unitPriceCents < 1 || unitPriceCents > MAX_UNIT_PRICE) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (quantity < 1 || quantity > MAX_QUANTITY) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }

        int subtotal = unitPriceCents * quantity;
        int discountedSubtotal;
        if (premiumCustomer) {
            discountedSubtotal = (int) (subtotal * PREMIUM_DISCOUNT / PERCENTAGE);
        } else {
            discountedSubtotal = subtotal;
        }
        if (discountedSubtotal >= FREE_DELIVERY) {
            return discountedSubtotal;
        }
        else {
            return discountedSubtotal + SHIPPING;
        }
    }
}
```

2. Грешките во Product.java:

Тука имаме **1 грешка** која што е **MagicNumber** и таа можеме да ја поправиме со повторно искористување на public static final променливата која што ја направивме во PriceCalculator.java:

PriceCalculator.*MAX_UNIT_PRICE*

При повторно извршување на тестовите можеме да видиме дека веќе нема грешки во оваа класа

```
[INFO] There are 2 errors reported by Checkstyle 10.21.2 with config/checkstyle.xml ruleset.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[4,8] (imports) UnusedImports: Unused import - java.util.ArrayList.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[29,40] (coding) MagicNumber: '100' is a magic number.
```

Кодот од класата **Product.java ПРЕД** промените:

```java
package mk.ukim.finki.testing.shop;

/** Product data returned by the catalog. Stock is a snapshot, not a reservation. */
public record Product(String code, int unitPriceCents, int availableQuantity) {  2 usages

    public Product {  2 usages
        if (code == null || code.isBlank()) {
            throw new IllegalArgumentException("Product code must not be blank");
        }
        if (unitPriceCents < 1 || unitPriceCents > 1_000_000) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (availableQuantity < 0) {
            throw new IllegalArgumentException("Available quantity must not be negative");
        }
    }
}
```

Кодот од класата **Product.java ПОСЛЕ** промените:

```java
package mk.ukim.finki.testing.shop;

/** Product data returned by the catalog. Stock is a snapshot, not a reservation. */
public record Product(String code, int unitPriceCents, int availableQuantity) {  2 usages

    public Product {  2 usages
        if (code == null || code.isBlank()) {
            throw new IllegalArgumentException("Product code must not be blank");
        }
        if (unitPriceCents < 1 || unitPriceCents > PriceCalculator.MAX_UNIT_PRICE) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (availableQuantity < 0) {
            throw new IllegalArgumentException("Available quantity must not be negative");
        }
    }
}
```

3. Грешките во OrderService.java:

Тука можеме да видиме дека имаме **1 грешка** од типот **UnusedImports**, која што може да се поправи со бришење на неискористениот import:
import java.util.ArrayList;

По бришењето на овој import и повторно извршување на командата **./mvnw -Pstatic-analysis validate**, можеме да видиме дека единствената грешка што останува е **MagicNumber**:

```
[INFO] There is 1 error reported by Checkstyle 10.21.2 with config/checkstyle.xml ruleset.
[ERROR] src/main/java/mk/ukim/finki/testing/shop/OrderService.java:[28,40] (coding) MagicNumber: '100' is a magic number.
```

Последната грешка е **MagicNumber** и таа можеме да ја поправиме со пак со повторно искористување на public static final променливата која што ја направивме во PriceCalculator.java:

PriceCalculator.*MAX_QUANTITY*

При повторно извршување на тестовите можеме да видиме дека веќе воопшто немаме грешки во сите класи

```
ognenmladenovski@Mac mini-shop % ./mvnw -Pstatic-analysis validate
[INFO] Scanning for projects...
[INFO]
[INFO] ------------< mk.ukim.finki.testing:unit-testing-homework >-------------
[INFO] Building Mini Shop - Static Analysis and Unit Testing Homework 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- checkstyle:3.6.0:check (check-source-quality) @ unit-testing-homework ---
[INFO] Starting audit...
Audit done.
[INFO] You have 0 Checkstyle violations.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.583 s
[INFO] Finished at: 2026-10-27T16:27:36+01:00
[INFO] ------------------------------------------------------------------------
```

Кодот од класата **OrderService.java ПРЕД** промените:

```java
package mk.ukim.finki.testing.shop;

import java.util.Objects;
import java.util.ArrayList;

public class OrderService {  no usages

    private final ProductRepository products;  2 usages
    private final OrderRepository orders;  2 usages
    private final OrderNotifier notifier;  2 usages
    private final PriceCalculator calculator;  2 usages

    public OrderService(ProductRepository products, OrderRepository orders,  1 usage
                        OrderNotifier notifier, PriceCalculator calculator) {
        this.products = Objects.requireNonNull(products);
        this.orders = Objects.requireNonNull(orders);
        this.notifier = Objects.requireNonNull(notifier);
        this.calculator = Objects.requireNonNull(calculator);
    }

    public Order placeOrder(String customerId, String productCode,  no usages
                            int quantity, boolean premiumCustomer) {
        if (customerId == null || customerId.isBlank()) {
            throw new IllegalArgumentException("Customer ID must not be blank");
        }
        if (productCode == null || productCode.isBlank()) {
            throw new IllegalArgumentException("Product code must not be blank");
        }
        if (quantity < 1 || quantity > 100) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }

        Product product = products.findByCode(productCode)
                .orElseThrow(() -> new ProductNotFoundException(productCode));
        if (quantity > product.availableQuantity()) {
            throw new InsufficientStockException(quantity, product.availableQuantity());
        }

        int total = calculator.calculateTotal(product.unitPriceCents(), quantity, premiumCustomer);
        Order order = new Order(customerId, productCode, quantity, total);
        orders.save(order);
        notifier.sendConfirmation(order);
        return order;
    }
}
```

Кодот од класата **OrderService.java ПОСЛЕ** промените:

```java
package mk.ukim.finki.testing.shop;

import java.util.Objects;

public class OrderService {  no usages

    private final ProductRepository products;  2 usages
    private final OrderRepository orders;  2 usages
    private final OrderNotifier notifier;  2 usages
    private final PriceCalculator calculator;  2 usages

    public OrderService(ProductRepository products, OrderRepository orders,  1 usage
                        OrderNotifier notifier, PriceCalculator calculator) {
        this.products = Objects.requireNonNull(products);
        this.orders = Objects.requireNonNull(orders);
        this.notifier = Objects.requireNonNull(notifier);
        this.calculator = Objects.requireNonNull(calculator);
    }

    public Order placeOrder(String customerId, String productCode,  no usages
                            int quantity, boolean premiumCustomer) {
        if (customerId == null || customerId.isBlank()) {
            throw new IllegalArgumentException("Customer ID must not be blank");
        }
        if (productCode == null || productCode.isBlank()) {
            throw new IllegalArgumentException("Product code must not be blank");
        }
        if (quantity < 1 || quantity > PriceCalculator.MAX_QUANTITY) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }

        Product product = products.findByCode(productCode)
                .orElseThrow(() -> new ProductNotFoundException(productCode));
        if (quantity > product.availableQuantity()) {
            throw new InsufficientStockException(quantity, product.availableQuantity());
        }

        int total = calculator.calculateTotal(product.unitPriceCents(), quantity, premiumCustomer);
        Order order = new Order(customerId, productCode, quantity, total);
        orders.save(order);
        notifier.sendConfirmation(order);
        return order;
    }
}
```

2. Unit тестирање

За класата **Product.java**, правиме фајл со тестови **ProductTest.java** и тука е потребно да се направат следните тестови:

1. **Тест за продукт со валидни вредности** (**createsProductWithGivenValues**) – во овој тест се креира нов продукт со **валидни вредности** и потоа се тестираат тие вредности дали се исти како доделените:

```
12      @Test
13      void createsProductWithGivenValues() {
14          Product product = new Product( code: "ValidCode",  unitPriceCents: 30,  availableQuantity: 10);
15
16          assertEquals( expected: "ValidCode", product.code());
17          assertEquals( expected: 30, product.unitPriceCents());
18          assertEquals( expected: 10, product.availableQuantity());
19      }
```

2. **Тест за одбивање на празен код** (**rejectsBlankCode**) – овој тест проверува дали имаме код со вредност: **null**, **празен**, **празно место**, **нов ред** и **tab**, доколку имаме се исфрла error пораката од претходно:

```
21      @ParameterizedTest
22      @NullAndEmptySource
23      @ValueSource(strings = {" ", "\n", "\t"})
24      void rejectsBlankCode(String code) {
25          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> new Product(code,  unitPriceCents: 35,  availableQuantity: 15));
26
27          assertEquals( expected: "Product code must not be blank", exception.getMessage());
28      }
```

3. **Тест за одбивање на цена надвор од дозволениот опсег** (**rejectsOutOfRangePrice**) – овој тест проверува 2 вредности за цената кои се надвор од опсегот (**1-1000000**) односно **0 која е помал од опсегот** и **1000001 кој е поголем од опсегот**, доколку бројот е надвор од опсегот се исфрла error пораката од претходно:

```
30      @ParameterizedTest
31      @ValueSource(ints = {0, 1_000_001})
32      void rejectsOutOfRangePrice(int price) {
33          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> new Product( code: "ValidCode", price,  availableQuantity: 20));
34
35          assertEquals( expected: "Unit price must be between 1 and 1000000 cents", exception.getMessage());
36      }
```

4. **Тест за одбивање на негативна количина** (**rejectsNegativeValueQuantity**) – овој тест проверува дали бројот за количина е негативен, -1 во овој случај, доколку бројот за количината е негативен се исфрла error пораката од претходно:

```
38      @Test
39      void rejectsNegativeValueQuantity(){
40          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> new Product( code: "ValidCode",  unitPriceCents: 45,  availableQuantity: -1));
41
42          assertEquals( expected: "Available quantity must not be negative", exception.getMessage());
43      }
```

За класата **PriceCalculator.java**, правиме фајл со тестови **PriceCalculatorTest.java** со променлива **calculator** која се употребува во сите тестови и таа се повикува пред секој тест со помош на методот **setUp()** кој што има анотација **@BeforeEach**. Тука е потребно да се направат следните тестови:

1. **Тест за одбивање на цена надвор од дозволениот опсег** (**rejectsOutOfRangePrice**) – овој тест проверува 2 вредности за цената кои се надвор од опсегот (**1-1000000**) односно **0 која е помал од опсегот** и **1000001 кој е поголем од опсегот**, доколку бројот е надвор од опсегот се исфрла error пораката од претходно:

```
@ParameterizedTest
@ValueSource(ints = {0, 1_000_001})
void rejectsOutOfRangePrice(int price) {
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> calculator.calculateTotal(price, quantity: 10, premiumCustomer: false));

    assertEquals( expected: "Unit price must be between 1 and 1000000 cents", exception.getMessage());
}
```

2. **Тест за одбивање количина надвор од дозволениот опсег** (**rejectsOutOfRangeQuantity**) – овој тест проверува 2 вредности за количината кои се надвор од опсегот (**1-100**) односно **0 која е помал од опсегот** и **101 кој е поголем од опсегот**, доколку бројот е надвор од опсегот се исфрла error пораката од претходно:

```
@ParameterizedTest
@ValueSource(ints = {0, 101})
void rejectsOutOfRangeQuantity(int quantity) {
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> calculator.calculateTotal( unitPriceCents: 100, quantity, premiumCustomer: false)

    assertEquals( expected: "Quantity must be between 1 and 100", exception.getMessage());
}
```

3. **Тест за пресметување Subtotal и Total цена** (**calculatesSubtotal**) – овој тест ја пресметува цената за **премиум купувач** со неговиот попуст (1000 * 2 = 2000 - 10% = 1800 + 500 = **2300**):

```
@Test
void calculatesSubtotal() {  //Both Subtotal and Total
    int total = calculator.calculateTotal( unitPriceCents: 1000, quantity: 2, premiumCustomer: true);

    assertEquals( expected: 2300, total);
}
```

4. **Тест за пресметување цена на стандарден купувач** (**calculatesStandardCustomer**) – овој тест ја пресметува цената за **стандарден купувач** без попуст (1000 * 6 = **6000**)

```
@Test
void calculatesStandardCustomer() {
    int total = calculator.calculateTotal( unitPriceCents: 1000, quantity: 6, premiumCustomer: false);

    assertEquals( expected: 6000, total);
}
```

5. **Тест за пресметување цена на премиум купувач** (**calculatesPremiumCustomer**) – овој тест ја пресметува цената за **премиум купувач** со попуст (515 * 1 = 515 – 10% = 463.5 + 500 = 963.5 ~ **963** (**заокружува на помала вредност**))

```
@Test
void calculatesPremiumCustomer() {
    int total = calculator.calculateTotal( unitPriceCents: 515, quantity: 1, premiumCustomer: true);

    assertEquals( expected: 963, total);
}
```

6. **Тест за пресметување цена со достава после попуст** (**calculatesShipping**) – овој тест ја пресметува цената за **премиум купувач** со **попуст + достава** (5000 * 1 = 5000 – 10% = 4500 + 500 = **5000**)

```
56      @Test
57      void calculatesShipping() {
58          int total = calculator.calculateTotal( unitPriceCents: 5000,  quantity: 1,  premiumCustomer: true);
59
60          assertEquals( expected: 5000, total);
61      }
```

7. **Тест за пресметување цена со бесплатна достава** (**calculatesFreeShipping**) – овој тест ја пресметува цената за **стандарден купувач** со **бесплатна достава** (5000 * 1 = **5000**)

```
63      @Test
64      void calculatesFreeShipping()
65      {
66          int total = calculator.calculateTotal( unitPriceCents: 5000,  quantity: 1,  premiumCustomer: false);
67
68          assertEquals( expected: 5000, total);
69      }
```

За класата **OrderService.java**, правиме фајл со тестови **OrderServiceTest.java**, тука користиме **Mockito** тестови бидејќи OrderService зависи од ProductRepository, OrderRepository и OrderNotifier од кои што **имаме само интерфејси а немаме имплементација**. Со анотацијата **@Mock** правиме mock објекти од нив. Пред секој тест во методот **setUp()** со анотација **@BeforeEach**, се креира нов **OrderService** со овие **mock објекти** и **вистински PriceCalculator**. Тука е потребно да се направат следните тестови:

1. **Тестови за валидирање на Customer ID прво, Product Code второ и Quantity трето** (**validatesCustomerIdFirst**, **validatesProductCodeSecond** и **validatesQuantityLast**) – овие 3 теста гарантираат дека проверка ќе се води во овој редослед, секоја порака исфрла error порака со соодветниот проблем:

```
36      @Test
37      void validatesCustomerIdFirst() {
38          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder( customerId: " ",  productCode: " ",  quantity: 0,  premiumCustomer: false));
39
40          assertEquals( expected: "Customer ID must not be blank", exception.getMessage());
41          verifyNoInteractions(products, orders, notifier);
42      }
43
44      @Test
45      void validatesProductCodeSecond() {
46          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder( customerId: "Customer1",  productCode: " ",  quantity: 0,  premiumCustomer: false));
47
48          assertEquals( expected: "Product code must not be blank", exception.getMessage());
49          verifyNoInteractions(products, orders, notifier);
50      }
51
52      @Test
53      void validatesQuantityLast() {
54          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder( customerId: "Customer1",  productCode: "ProductCode1",  quantity: 0,  premiumCustomer: false));
55
56          assertEquals( expected: "Quantity must be between 1 and 100", exception.getMessage());
57          verifyNoInteractions(products, orders, notifier);
58      }
```

2. **Тест за одбивање на празен Customer ID** (**rejectsBlankCustomerId**) – овој тест проверува дали имаме **Customer ID** со вредност: **null**, **празен**, **празно место**, **нов ред** и **tab**, доколку имаме се исфрла error пораката од претходно:

```
60      @ParameterizedTest
61      @NullAndEmptySource
62      @ValueSource(strings = {" ", "\n", "\t"})
63      void rejectsBlankCustomerId(String customerId) {
64          IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder(customerId,  productCode: "ProductCode1",  quantity: 1,  premiumCustomer: false));
65
66          assertEquals( expected: "Customer ID must not be blank", exception.getMessage());
67          verifyNoInteractions(products, orders, notifier);
68      }
```

3. **Тест за одбивање на празен Product Code** (**rejectsBlankProductCode**) – овој тест проверува дали имаме **Product Code** со вредност: **null**, **празен**, **празно место**, **нов ред** и **tab**, доколку имаме се исфрла error пораката од претходно:

```
@ParameterizedTest
@NullAndEmptySource
@ValueSource(strings = {" ", "\n", "\t"})
void rejectsBlankProductCode(String productCode) {
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder( customerId: "Customer1", productCode, quantity: 1, premiumCustomer: false));

    assertEquals( expected: "Product code must not be blank", exception.getMessage());
    verifyNoInteractions(products, orders, notifier);
}
```

1. **Тест за одбивање количина надвор од дозволениот опсег** (**rejectsOutOfRangeQuantity**) – овој тест проверува 2 вредности за количината кои се надвор од опсегот (**1-100**) односно **0 која е помал од опсегот** и **101 кој е поголем од опсегот**, доколку бројот е надвор од опсегот се исфрла error пораката од претходно:

```
@ParameterizedTest
@ValueSource(ints = {0, 101})
void rejectsOutOfRangeQuantity(int quantity) {
    IllegalArgumentException exception = assertThrows(IllegalArgumentException.class, () -> service.placeOrder( customerId: "Customer1", productCode: "ProductCode1", quantity, premiumCustomer: false));

    assertEquals( expected: "Quantity must be between 1 and 100", exception.getMessage());
    verifyNoInteractions(products, orders, notifier);
}
```

4. **Тест за барање на продукт по точниот код само еднаш** (**looksUpProductByExactCodeOnlyOnce**) – во овој тест со **when(...).thenReturn(...)** му кажуваме на mock објектот products дека кога ќе се побара кодот „ ProductCode1 “ (со празни места), треба да врати продукт со тој код. Потоа се прави нарачка и со **verify(products, times(1))** се проверува дека продуктот е побаран **точно еднаш со истиот код**, без да се избришат празните места, и се прави проверка дали нарачката ги содржи **Customer ID** и **Product Code** точно како што се дадени:

```
@Test
void looksUpProductByExactCodeOnlyOnce() {
    when(products.findByCode(" ProductCode1 ")).thenReturn(Optional.of(new Product( code: " ProductCode1 ", unitPriceCents: 1000, availableQuantity: 10)));

    Order order = service.placeOrder( customerId: "Customer1", productCode: " ProductCode1 ", quantity: 1, premiumCustomer: false);

    verify(products, times( wantedNumberOfInvocations: 1)).findByCode(" ProductCode1 ");
    assertEquals( expected: "Customer1", order.customerId());
    assertEquals( expected: " ProductCode1 ", order.productCode());
}
```

5. **Тест за одбивање на непостоечки продукт** (**rejectsUnknownProduct**) – во овој тест со **when(...).thenReturn(Optional.empty())** му кажуваме на mock објектот products дека **продуктот не постои**. Затоа се исфрла **ProductNotFoundException** со пораката „Product not found: ProductCode1“:

```
@Test
void rejectsUnknownProduct() {
    when(products.findByCode("ProductCode1")).thenReturn( t: Optional.empty());

    ProductNotFoundException exception = assertThrows(ProductNotFoundException.class, () -> service.placeOrder( customerId: "Customer1", productCode: "ProductCode1", quantity: 1, premiumCustomer: false));

    assertEquals( expected: "Product not found: ProductCode1", exception.getMessage());
    verifyNoInteractions(orders, notifier);
}
```

6. **Тест за одбивање на количина поголема од залихата** (**rejectsQuantityAboveStock**) – во овој тест со **when(...).thenReturn(...)** му кажуваме на mock објектот products да врати продукт со **залиха 5**, а нарачката бара **6 парчиња**. Затоа се исфрла **InsufficientStockException** со пораката „Requested quantity 6 exceeds available quantity 5“:

```
@Test
void rejectsQuantityAboveStock() {
    when(products.findByCode("ProductCode1")).thenReturn(Optional.of(new Product( code: "ProductCode1",  unitPriceCents: 1000,  availableQuantity: 5)));

    InsufficientStockException exception = assertThrows(InsufficientStockException.class, () -> service.placeOrder( customerId: "Customer1",  productCode: "ProductCode1",  quantity: 6,  premiumCustomer: false));

    assertEquals( expected: "Requested quantity 6 exceeds available quantity 5", exception.getMessage());
    verifyNoInteractions(orders, notifier);
}
```

7. **Тест за проверка на успешна нарачка** (**placesOrder**) – во овој тест со **when(...).thenReturn(...)** му кажуваме на mock објектот products да врати продукт со **залиха 5**, а нарачката бара точно **5 парчиња**, што е дозволено. Потоа се проверува дека вратената нарачка ги содржи точните **Customer ID**, **Product Code**, **количина** и **вкупна цена** (1000 * 5 = 5000, без достава). Со **InOrder** се проверува дека нарачката прво е зачувана, а потоа е пратена потврда, со истата нарачка и точно еднаш:

```
@Test
void placesOrder() {
    when(products.findByCode("ProductCode1")).thenReturn(Optional.of(new Product( code: "ProductCode1",  unitPriceCents: 1000,  availableQuantity: 5)));

    Order order = service.placeOrder( customerId: "Customer1",  productCode: "ProductCode1",  quantity: 5,  premiumCustomer: false);

    assertEquals( expected: "Customer1", order.customerId());
    assertEquals( expected: "ProductCode1", order.productCode());
    assertEquals( expected: 5, order.quantity());
    assertEquals( expected: 5000, order.totalCents());

    InOrder inOrder = inOrder(orders, notifier);
    inOrder.verify(orders).save(same(order));
    inOrder.verify(notifier).sendConfirmation(same(order));
    verifyNoMoreInteractions(orders, notifier);
}
```

8. **Тест за проверка на грешка при зачувување** (**checksSaveFailure**) – во овој тест со **when(...).thenReturn(...)** му кажуваме на mock објектот products да врати продукт, а со **doThrow(...)** правиме mock објектот orders што фрла **IllegalStateException** при зачувување. Се проверува дека истата грешка се пренесува понатаму и со **verifyNoInteractions(notifier)** се проверува дека не е пратена потврда:

```
@Test
void checksSaveFailure() {
    when(products.findByCode("ProductCode1")).thenReturn(Optional.of(new Product( code: "ProductCode1",  unitPriceCents: 1000,  availableQuantity: 5)));
    IllegalStateException failure = new IllegalStateException("Save failed");
    doThrow(failure).when(orders).save(any());

    IllegalStateException exception = assertThrows(IllegalStateException.class, () -> service.placeOrder( customerId: "Customer1",  productCode: "ProductCode1",  quantity: 1,  premiumCustomer: false));

    assertSame(failure, exception);
    verifyNoInteractions(notifier);
}
```

9. **Тест за грешка при праќање потврда** (**checksNotificationFailure**) – во овој тест со **when(...).thenReturn(...)** му кажуваме на mock објектот products да врати продукт, а со **doThrow(...)** правиме mock објектот notifier да фрли **IllegalStateException** при праќање потврда. Се проверува дека истата грешка се пренесува понатаму и со **verify(orders).save(...)** се проверува дека нарачката веќе е зачувана:



```
149     @Test
150     void checksNotificationFailure() {
151         when(products.findByCode("ProductCode1")).thenReturn(Optional.of(new Product( code: "ProductCode1",  unitPriceCents: 1000,  availableQuantity: 5)));
152         IllegalStateException failure = new IllegalStateException("Notification failed");
153         doThrow(failure).when(notifier).sendConfirmation(any());
154
155         IllegalStateException exception = assertThrows(IllegalStateException.class, () -> service.placeOrder( customerId: "Customer1",  productCode: "ProductCode1",  quantity: 1,  premiumCustomer: false));
156
157         assertSame(failure, exception);
158         verify(orders).save(any());
159     }
```

Студент: Огнен Младеновски 233108